

Formal Verification Documentation

Two-Party Cascade Protocol Security Checker

<https://github.com/KrishGarg/lean-security-protocol/>

Overview

This project is a formal verification tool and security checker written in Lean 4. It implements the security decision procedures for **algebraic cascade protocols** as defined in the Dolev-Even paper (specifically Theorems 6.2.6 and 6.2.8).

The application is capable of statically analyzing an instantiated protocol's transmission sequences (α_i and β_j) to determine in linear time whether it is secure against an active saboteur under both symmetric and nonsymmetric semantics. The codebase is fully modularized into a multi-file architecture to separate data types, semantics, decision procedures, and proofs.

1 Core Types (`Basic.lean`)

The foundational data structures model the cryptographic alphabet and sequence compositions.

- **User**: An inductive type representing the communicating parties. Currently constrained to X (the initiator) and Y (the receiver) for the scope of the two-party theorems.
- **Func**: An inductive type representing the primitive cryptographic operators.
 - $E\ u$: Encryption keyed to user u .
 - $D\ u$: Decryption keyed to user u .
- **Word**: An alias for `List Func`. It represents a composed sequence of cryptographic functions (e.g., $E_X D_Y E_X$).
- **Protocol**: An alias for `List Word  $\times$  List Word`, representing the finite sequences α_i and β_j that define the cascade protocol's message exchanges.

2 Rewriting & Reduction Semantics (`Reduction.lean`)

These functions handle the string-rewriting systems (T_1 and T_2) that define when adjacent cryptographic operations cancel each other out.

- `cancelsSymmetric (f g : Func) : Bool`: Implements T_1 semantics. Returns `true` if f and g are opposing functions for the same user (both $D_X \circ E_X = 1$ and $E_X \circ D_X = 1$).
- `cancelsNonsymmetric (f g : Func) : Bool`: Implements T_2 semantics. Returns `true` only for decrypt-then-encrypt sequences ($D_X \circ E_X = 1$).

- `reduceOnce (cancel) (w : Word) : Option Word`: Scans the word left-to-right and collapses the *first* valid adjacent cancellation it finds, returning `some` new word, or `none` if it is irreducible.
- `reduceAux (cancel) (fuel) (w) : Word`: A functional powerhouse that iteratively applies `reduceOnce` until no cancellations remain. Uses the initial word's length as `fuel` to guarantee termination, sidestepping Lean's `partial` keyword requirement.
- `reduceSymmetric / reduceNonsymmetric`: Helper wrappers that apply `reduceAux` with the respective cancellation rules.

3 Cryptographic Predicates (`Predicates.lean`)

These helper functions analyze the presence and arrangement of operators within a `Word` ($lt(\gamma)$ in the text).

- `hasEncryption / hasDecryption`: Fast boolean checks that scan a word to see if it contains *any* encryption or decryption operation, respectively.
- `containsFunc (w) (f)`: Checks if a specific exact function (e.g., D_X) exists in the word.
- `hasBalancingProperty (w) (u)`: The core invariant of a cascade protocol. It checks that if a decryption operator for user u (D_u) is present, an encryption operator for user u (E_u) must also be present in the sequence. This ensures an adversary cannot completely strip away a user's encryption.
- `containsAdjacentPair (w) (f) (g)`: Scans for exact adjacent sequence factors (e.g., $D_X E_Y$), a specific bypass condition required in nonsymmetric security checks.

4 Security Decision Procedures (`Security.lean`)

The main computable algorithms that yield a `true/false` verdict on the raw security of a protocol. (Note: See Section 5 for guarded entry points).

- `isSecureSymmetric (p : Protocol)`: Implements **Theorem 6.2.6**. Evaluates to `true` iff:
 1. α_1 contains at least one encryption operator.
 2. Every subsequent α_i is balanced with respect to X .
 3. Every β_j is balanced with respect to Y .
- `isSecureNonsymmetric (p : Protocol)`: Implements **Theorem 6.2.8**. Evaluates to `true` iff:
 1. α_1 contains a decryption operator, OR
 2. α_1 contains an encryption operator AND subsequent α_i / β_j sequences either satisfy their respective balancing properties *or* contain specific adjacent operator factors ($D_X E_Y$ for α , $D_Y E_X$ for β).

5 Input Validation (`Validation.lean`)

According to Definitions 6.1.2 and 6.1.3 of the Dolev-Even paper, all primitive operators $\alpha_i(X, Y)$ and $\beta_j(X, Y)$ in a valid protocol are assumed to be *already irreducible*. Our system actively enforces this before evaluating security.

- `isIrreducible (w : Word)`: Runs the reduction engine over a single word and verifies that the output is identical to the input.
- `allWordsIrreducible (p : Protocol)`: Iterates over all α and β sequences in a protocol to verify global irreducibility.
- `checkSecureSymmetric / checkSecureNonsymmetric`: Guarded wrapper functions that serve as the main entry points for the tool. These functions first pass the protocol through `allWordsIrreducible`. If the protocol contains reducible words, the system actively rejects it as invalid rather than yielding a false security verdict.

6 Examples & Theorems (`Examples.lean` & `Theorems.lean`)

- `Examples.lean`: Contains instantiated protocols (like the textbook example) used for testing and validating the executable components.
- `Theorems.lean`: Contains the mathematical stubs mapping the computable boolean logic to formal Lean 4 propositions.

7 Scope of Improvement

While the computational side of the checker is fully functional, the project can be evolved in the following ways.

A. Proving the Correctness Theorems (`Theorems.lean`)

The ultimate goal of formalizing this in Lean 4 is to replace the sorry stubs with actual mathematical proofs.

- Prove `balancing_correct` to map the computable boolean logic to Propositional Logic.
- Prove `symmetric_cascade_secure_iff` to formally bind the `isSecureSymmetric` function to the exact characterization of Theorem 6.2.6.
- Prove `reduceSymmetric_sound` to show that the length-fueled reduction engine perfectly preserves word semantics and conforms to confluence.

B. Inverse String Generation for Insecure Protocols

Currently, the tool simply yields `false` when a protocol is determined to be insecure. A powerful future enhancement would be to constructively demonstrate *why* it is insecure. By analyzing the failed balancing properties or missing operators, the system could computationally generate and output the exact “inverse string” (the sabotage sequence from $\Gamma_Z \cup \{\alpha, \beta\}^*$) that a saboteur would use to successfully invert the encryption and compromise the protocol.